

LAB 06

High-Level Design, Low-Level Design, UML, BPMN & Database Design

2. Scenario: University Facility Booking System

Students and lecturers can submit facility booking requests for classrooms, meeting rooms, and laboratories. A requester selects a facility, booking date, start time, end time, purpose, and expected number of participants. The system checks basic input validity and records the request as Pending. Staff members review pending requests, check facility availability and service rules, then approve or reject each request. The system sends a notification to the requester after submission and after the staff decision.

Each facility has a facility ID, facility name, location, capacity, facility type, and status. A booking request has a request ID, requester, facility, booking date, start time, end time, purpose, participant count, request status, and submission time. A staff decision records who reviewed the request, decision date, decision result, and decision note. Approved requests become booking records. The office wants to keep request history, avoid overlapping bookings for the same facility and time period, and support basic reporting on booking requests by date, facility, status, and requester type.

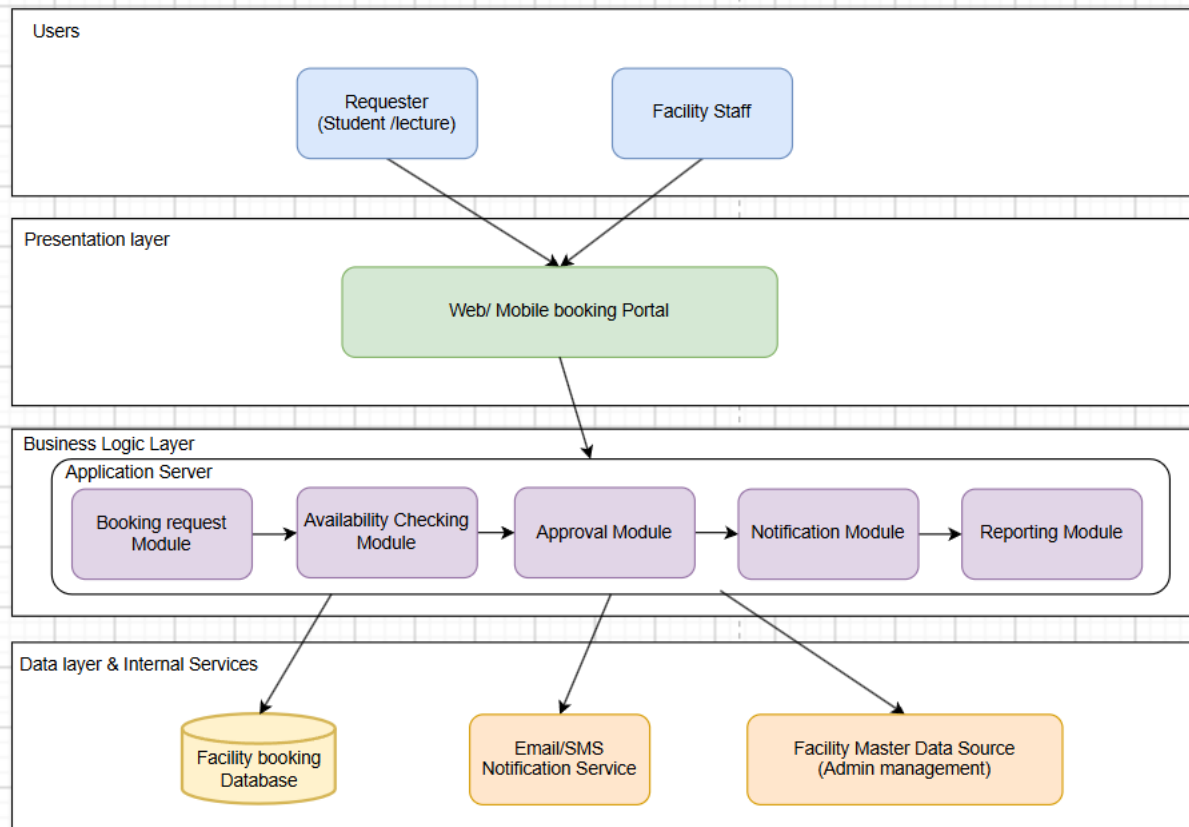
For this lab, students should focus on analysis-to-design consistency. The same case should be used across HLD, LLD, UML, BPMN, and database design.

3. Requirements

Complete the following tasks. This lab is focused on HLD, LLD, UML, BPMN, and Database Design.

Task 1 - High-Level Design

Prepare a High-Level Design for the University Facility Booking System. HLD should show the big-picture structure of the system, not the detailed workflow steps.



Major Modules / Components and Responsibilities

Module	Responsibility	Input	Output
Booking Request Module	Create and manage facility booking requests; record initial status as Pending.	RequesterID, FacilityID, BookingDate, StartTime, EndTime, Purpose, ParticipantCount	RequestID, RequestStatus
Availability Checking Module	Check facility availability and detect overlapping bookings for a given date/time.	FacilityID, BookingDate, StartTime, EndTime	AvailabilityStatus, ConflictingBookingID
Approval Module	Support staff review, apply service rules, approve/reject requests, and create booking records.	RequestID, StaffID, DecisionResult, DecisionNote	RequestStatus, BookingID
Notification Module	Send a notification to the requester after submission and after the staff decision.	RequesterID, RequestID, Status, MessageContent	NotificationStatus
Reporting Module	Generate booking reports by date, facility, status, and requester type.	Report filters (date range, facility, status)	Report data / export
Facility Management Module	Maintain facility master data (ID, name, location, capacity, type, status).	FacilityID, FacilityName, Location, Capacity, FacilityType	Facility record confirmation

Module Communication

Users access the system only through the Web/Mobile Booking Portal (presentation layer); they never access the database directly. The Booking Request Module receives new requests from the portal and stores them with status Pending. When a Facility Staff member opens a request in the Approval Module, the Approval Module calls the Availability Checking Module to verify that the requested facility, date, and time do not overlap with an existing

approved booking. After a decision is made, the Approval Module updates the request status, creates a booking record when approved, and calls the Notification Module so the requester is informed. The Reporting Module reads data independently to produce booking reports, and the Facility Management Module maintains the facility master data used by the other modules.

Major Data Groups and External Systems

Major data groups: Facility data (facility ID, name, location, capacity, type, status), Booking Request data (request ID, requester, facility, date/time, purpose, participant count, status, submission time), Staff Decision data (reviewer, decision date, result, note), and Booking data (confirmed bookings). External systems: an Email/SMS Notification Service is used to deliver submission and decision notifications to requesters.

Justification

This layered design is suitable because it keeps the booking business rules (availability checking, approval workflow) inside the Business Logic Layer, separated from the user interface and the database. It matches the scenario requirement to avoid overlapping bookings, since the Availability Checking Module verifies this centrally before any request is approved. It also supports the required reporting by date, facility, status, and requester type through a dedicated Reporting Module, and keeps notification delivery decoupled through an external service so it can be scaled or replaced independently of the core booking logic.

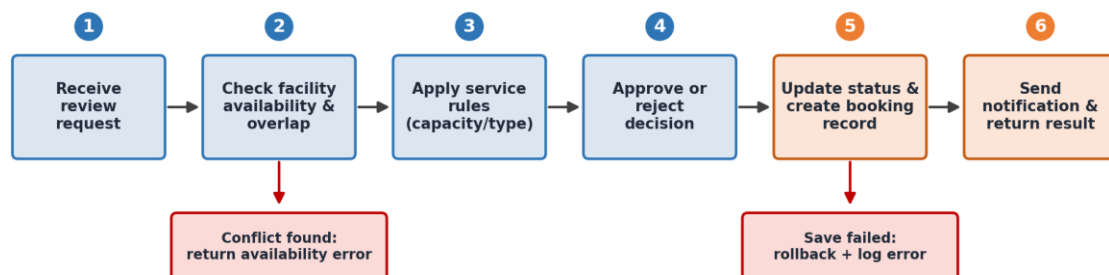
Task 2 - Low-Level Design

Prepare a Low-Level Design for one selected component. Recommended component: Booking Approval Component. This component supports staff review, availability checking, approval/rejection, booking creation, status update, and notification

1. Component Specification

Design Item	Content
Component Name	Booking Approval Component
Purpose	Support staff review of pending booking requests, check facility availability, approve or reject requests, create booking records, update request status, and trigger notification.
Related Use Case	Approve Facility Booking Request
Input Data	RequestID, StaffID, DecisionResult, DecisionNote
Output Data	RequestStatus, BookingID (if approved), NotificationStatus
Data Stores / Entities Used	FACILITY, BOOKING REQUEST, STAFF DECISION, BOOKING
Other Components Called	Availability Checking Module, Notification Module

2. Internal Flow



The internal flow shows six main steps inside the component: (1) receive the review request from staff, (2) check facility availability and overlap with existing approved bookings, (3) apply service rules such as capacity and facility-type match, (4) record the staff approve/reject decision, (5) update the request status and create a booking record when approved, and (6) send the notification and return the result to the staff interface. A conflict found in step 2 and a save failure in step 5 are handled as explicit error branches instead of silently terminating the component.

3. Processing Rules

ID	Condition	System Action	Design Implication
PR-01	Staff opens a request with RequestStatus = Pending	Load request, facility, and requester details for review	Only pending requests can be reviewed; approved/rejected requests are read-only
PR-02	Selected facility already has an approved booking overlapping the requested date/time	Block the approval and return a conflict message	Prevents double-booking of the same facility and time slot
PR-03	Staff selects Approve and no conflict is found	Create a Booking record and set RequestStatus = Approved	Booking creation rule; a booking always originates from an approved request
PR-04	Staff selects Reject	Set RequestStatus = Rejected and require a DecisionNote	Status transition rule; a reason must be recorded for rejection
PR-05	A staff decision (Approve or Reject) has been saved	Call the Notification Module through the interface contract to inform the requester	Interface contract call; keeps notification logic outside the Approval Module
PR-06	RequestStatus is already Approved or Rejected	Prevent staff from submitting a new decision on the same request	Terminal business outcome; enforces a valid status transition

4. Validation Rules

ID	Field / Condition	Validation Rule	Error Message	Where Enforced
V1	DecisionResult	Must be Approve or Reject; cannot be empty	"Please select Approve or Reject."	Approval Module
V2	DecisionNote	Required when DecisionResult = Reject	"Please provide a reason for rejection."	Approval Module
V3	RequestID	Must exist and have RequestStatus = Pending	"This request is no longer pending."	Approval Module
V4	StaffID	Must belong to an authenticated Facility Staff account	"Access denied. Please log in as facility staff."	Authentication layer
V5	FacilityID + BookingDate + StartTime + EndTime	Must not overlap an existing approved booking for the same facility	"Selected time slot is already booked."	Availability Checking Module
V6	ParticipantCount	Must not exceed the facility's Capacity	"Number of participants exceeds facility capacity."	Approval Module

5. Interface Contract

Item	Specification
Operation	checkFacilityAvailability(facilityId, bookingDate, startTime, endTime)
Source	Approval Module
Target	Availability Checking Module
Input	FacilityID, BookingDate, StartTime, EndTime
Output	AvailabilityStatus (Available / Conflict), ConflictingBookingID (if any)
Precondition	RequestStatus = Pending and the facility exists
Postcondition	An availability result is returned; no booking data is changed
Error Cases	Facility not found; invalid time range (EndTime is earlier than or equal to StartTime)
Security	Only authenticated internal modules can call this operation

6. Error Handling

Code	Error Case	User Message	System Action	Result
E01	Request already decided	"This request has already been processed."	Reject the new decision attempt; keep existing status	Stop
E02	Facility conflict detected	"Selected time slot is already booked. Please choose another slot or reject the request."	Keep RequestStatus = Pending; log the conflict	Stop
E03	Database save failure when creating booking / updating status	"The decision could not be saved. Please try again."	Roll back the transaction; log the error with a trace ID; alert support	Rollback
E04	Notification send failure	"Decision saved, but notification delivery is pending."	Save the notification as Pending/Failed and retry later	Continue

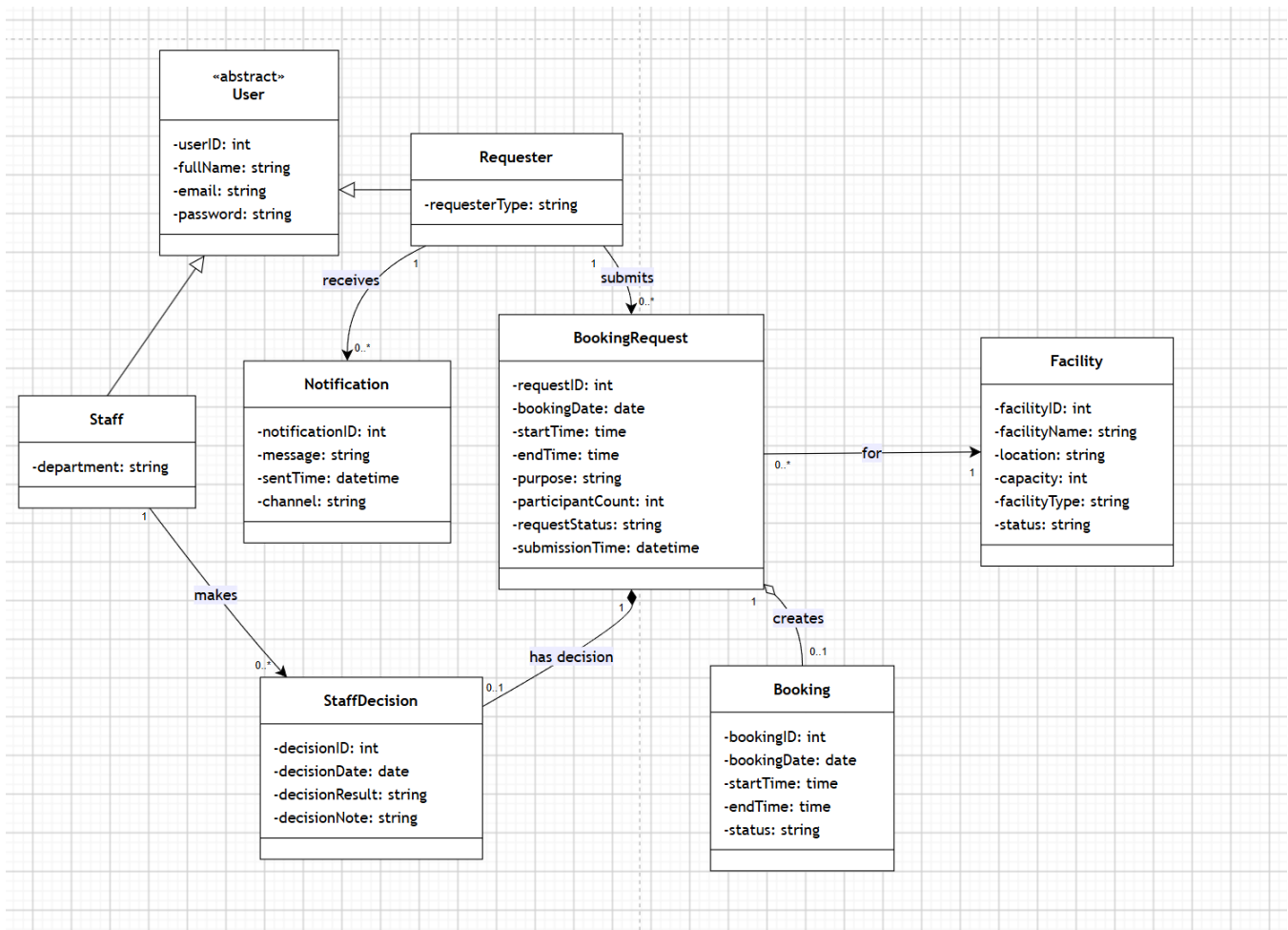
7. Controls

Control	Example in the Booking Approval Component
Authentication	Facility staff must log in with a valid staff account before reviewing requests.
Authorization	Only the Facility Staff role can approve or reject; a requester cannot approve their own request.
Audit Log	Store StaffID, DecisionDate, DecisionResult, and DecisionNote for every decision.
Data Integrity	Foreign key constraints between BOOKING_REQUEST, FACILITY, STAFF_DECISION, and BOOKING; the availability check is enforced before insert.

Task 3 - UML Modeling

Create UML diagrams that support the design of the University Facility Booking System. The diagrams must be consistent with the HLD and LLD decisions.

Class Diagram



The class diagram includes eight domain classes: User (abstract), Requester, Staff, Facility, BookingRequest, StaffDecision, Booking, and Notification. Requester and Staff inherit from the abstract User class (generalization). BookingRequest has a composition relationship with StaffDecision, since a decision cannot exist without the request it belongs to, and an aggregation relationship with Booking, since a booking record is created from an approved request but represents a separate, independently referenced object.

Sequence Diagram - Approve Facility Booking Request

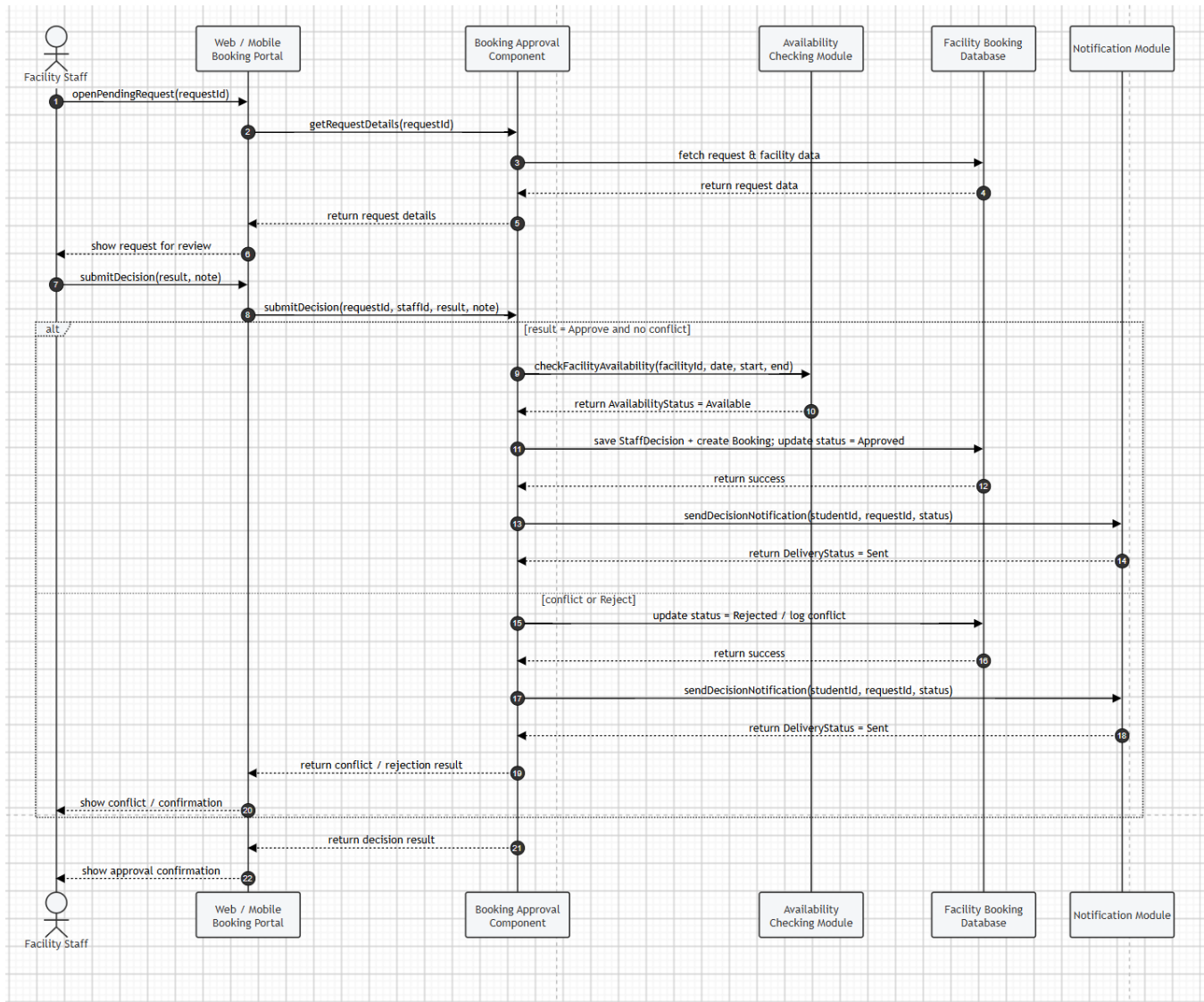


Figure 4. UML Sequence Diagram - Approve Facility Booking Request

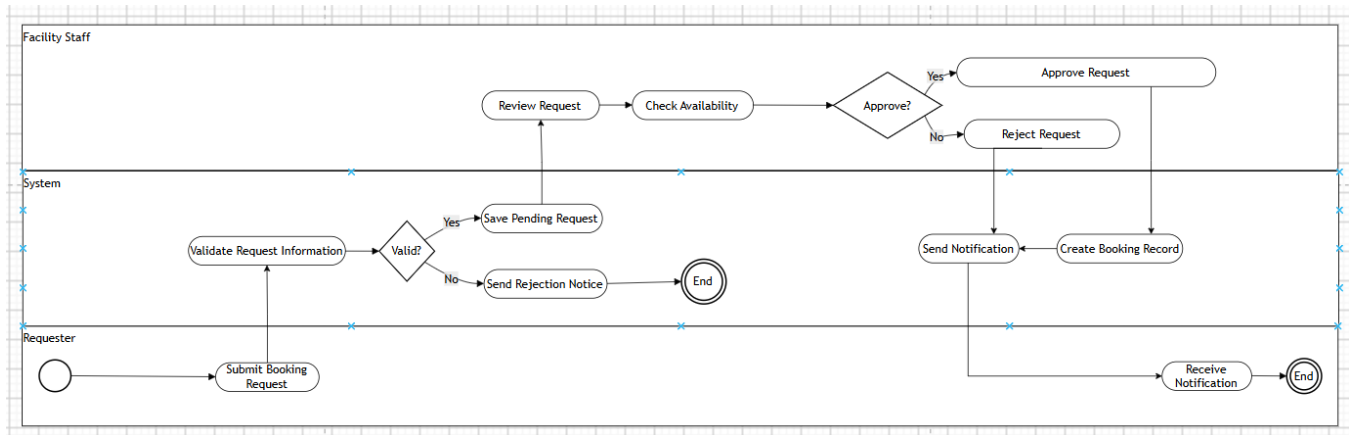
The sequence diagram shows Facility Staff, the Web/Mobile Booking Portal, the Booking Approval Component, the Availability Checking Module, the Facility Booking Database, and the Notification Module. After staff submit a decision, the alt fragment separates two cases:

1. If the request is approved and no conflict exists, the system creates a booking record, updates the status, and sends a notification to the requester.
2. If a conflict is found or the request is rejected, the system updates the status and logs the decision without creating a booking, but still sends a notification to inform the requester before returning the final result to the staff interface.

Task 4 - Draw BPMN workflow

Draw a BPMN diagram for Facility Booking Approval.

The BPMN diagram should include:



The BPMN diagram uses three lanes inside a single pool (Requester, System, Facility Staff). The Requester submits a booking request; the System validates the request and, if invalid, sends a rejection notice and the process ends. If valid, the request is saved with status Pending and routed to Facility Staff for review. Facility Staff check availability and either approve or reject the request; approved requests trigger the System to create a booking record before sending the notification, while rejected requests go straight to the notification step. The Requester receives the notification and the process ends.

Task 5 - Map data model to logical tables

Map the main domain classes or Concept ERD elements to logical tables. Identify primary keys, foreign keys, important attributes, and the business purpose of each table.

Answer

Table Name	Primary Key	Foreign Key(s)	Important Attributes
FACILITY	FacilityID	-	FacilityName, Location, Capacity, FacilityType, Status
USER_ACCOUNT	UserID	-	FullName, Email, Password, Role (Requester/Staff), RequesterType, Department
BOOKING_REQUEST	RequestID	RequesterID (UserID), FacilityID	BookingDate, StartTime, EndTime, Purpose, ParticipantCount, RequestStatus, SubmissionTime
STAFF_DECISION	DecisionID	RequestID, StaffID (UserID)	DecisionDate, DecisionResult, DecisionNote
BOOKING	BookingID	RequestID, FacilityID	BookingID, RequestID, FacilityID và Status
NOTIFICATION	NotificationID	RequesterID (UserID), RequestID	Message, SentTime, Channel

Business Purpose of Each Table

- FACILITY - stores facility master data used for booking and availability checking.
- USER_ACCOUNT - stores login and identity data for both requesters and facility staff.
- BOOKING_REQUEST - records each booking request submitted by a requester and its current status.
- STAFF_DECISION - records the staff review decision (approve/reject) for one booking request.
- BOOKING - stores confirmed bookings that are created after a request is approved.

- NOTIFICATION - stores notifications sent to requesters after submission and after the staff decision.

Task 6- Identify constraints and business rules

Write database-related constraints and business rules that should be enforced or checked. Some rules may be implemented by database constraints, while others may require application logic.

Rule / Constraint	Related Table(s)	Reason
A booking request cannot be approved if the facility already has an approved booking overlapping the same date and time.	BOOKING_REQUEST, BOOKING	Prevents double-booking of the same facility and time slot.
RequestStatus must follow a valid transition: Pending -> Approved / Rejected only.	BOOKING_REQUEST	Keeps request status meaningful and auditable.
ParticipantCount must not exceed the Capacity of the selected facility.	BOOKING_REQUEST, FACILITY	Ensures the facility can physically accommodate the booking.
Every STAFF_DECISION record must reference a valid, existing BOOKING_REQUEST (foreign key constraint).	STAFF_DECISION, BOOKING_REQUEST	A decision cannot exist without the request it belongs to (data integrity).
A BOOKING record can only be created after a request has been Approved.	BOOKING, BOOKING_REQUEST	Bookings must always originate from an approved request, not directly from a pending one.
BookingDate, FacilityID, and RequestStatus should be indexed to support reporting by date, facility, status, and requester type.	BOOKING_REQUEST	Improves performance of the required booking reports.

4. Assessment (100 points total)

Task #	Criteria	Points
1	HLD	10
2	LLD	15
3	UML modeling	25
4	BPMN workflow	20
5	Database design	20
6	Business rules and constraints	10

5. Submission

Submit one PDF report and editable diagram/database design file(s). File name format: CSW309_Lab06_StudentNo_StudentName.

6. Notice for Student

- BPMN models the business process; it should not be identical to a sequence diagram.
- Database design should be based on entities/classes and business rules, not on screen layout.